最小生成树选讲

wzj52501

Peking University

2019 年 7 月 28 日

MST 问题概述

- 在一给定的无向图 $G = (V, E)$ 中，(u, v) 代表连接结点 u 与结点 v 的边，而 $w(u, v)$ 代表此边的权重。

MST 问题概述

- 在一给定的无向图 $G = (V, E)$ 中，(u, v) 代表连接结点 u 与结点 v 的边，而 $w(u, v)$ 代表此边的权重。
- 一个有 n 个结点的连通图的生成树是原图的极小连通子图，且包含原图中的所有 n 个结点并有保持图连通的最少的边。

MST 问题概述

- 在一给定的无向图 $G = (V, E)$ 中，(u, v) 代表连接结点 u 与结点 v 的边，而 $w(u, v)$ 代表此边的权重。
- 一个有 n 个结点的连通图的生成树是原图的极小连通子图，且包含原图中的所有 n 个结点并有保持图连通的最少的边。
- 定义一棵生成树 T 的权值 $W(T) = \sum_{(u,v)\in T} w(u, v)$。

MST 问题概述

- 在一给定的无向图 $G = (V, E)$ 中，(u, v) 代表连接结点 u 与结点 v 的边，而 $w(u, v)$ 代表此边的权重。
- 一个有 n 个结点的连通图的生成树是原图的极小连通子图，且包含原图中的所有 n 个结点并有保持图连通的最少的边。
- 定义一棵生成树 T 的权值 $W(T) = \sum_{(u,v)\in T} w(u, v)$。
- 最小生成树 MST 即为 $W(T)$ 最小的 T。

Kruskal 算法

- 按照 $w(u, v)$ 从小到大将边集排序，然后顺序扫描每一条边 (u, v)。
- 如果 u 和 v 在不同的连通分量就将边 (u, v) 加入到最小生成树中。
- 使用并查集维护连通信息，时间复杂度为 $O(m \log m + m\alpha(n))$。
- 正确性可以结合拟阵证明。

Prim 算法

- Step1：将任意结点加入集合 S。
- Step2：找到最小的 $w(u, v)$ 满足 $u \in S$ 且 $v \notin S$。
- Step3：将 v 加入集合 S，将 (u, v) 加入最小生成树中。
- Step4：重复 Step2、Step3 直到集合 S 包含所有结点。

Prim 算法

- Step1：将任意结点加入集合 S。
- Step2：找到最小的 $w(u, v)$ 满足 $u \in S$ 且 $v \notin S$。
- Step3：将 v 加入集合 S，将 (u, v) 加入最小生成树中。
- Step4：重复 Step2、Step3 直到集合 S 包含所有结点。

时间复杂度为 $O(n^2)$，正确性可由 Kruskal 算法推出。

MST 常规算法

我们可以看出两种算法分别适用于稀疏图和稠密图的情况，可以解决大部分的最小生成树问题。

例题 1

货车运输 (NOIP2013)

A 国有 n 座城市，编号从 1 到 n，城市之间有 m 条双向道路。每一条道路对车辆都有重量限制，简称限重。现在有 q 辆货车运输货物，第 i 辆货车将从 x_i 城市运输货物到 y_i 城市。司机们想知道每辆车在不超过车辆限重的情况下，最多能运多重的货物。

$n \leq 10000, m \leq 50000, q \leq 30000$

例题 1

货车运输 (NOIP2013)

A 国有 n 座城市，编号从 1 到 n，城市之间有 m 条双向道路。每一条道路对车辆都有重量限制，简称限重。现在有 q 辆货车运输货物，第 i 辆货车将从 x_i 城市运输货物到 y_i 城市。司机们想知道每辆车在不超过车辆限重的情况下，最多能运多重的货物。

$n \leq 10000, m \leq 50000, q \leq 30000$

- 题目所求即为若干点对间的最大瓶颈路，而使用 Kruskal 算法求出的最大生成树即为最大瓶颈树。
- 故进行 Kruskal 算法求出最大瓶颈树后，使用 ST 表等算法维护树上路径最值即可。
- 时间复杂度为 $O(m \log m + (q+n) \log n)$。

例题 2

Tree

给定一个 n 个结点 m 条带权边的无向连通图，对于每一条边求出强制包含该边的最小生成树的权值。

$n, m \leq 10^6$

例题 2

Tree

给定一个 n 个结点 m 条带权边的无向连通图，对于每一条边求出强制包含该边的最小生成树的权值。

$n, m \leq 10^6$

- 先求出原图的最小生成树 T，考虑强制加入边 $e_1 = (u_1, v_1, w_1)$ 后会有怎样的变化。

例题 2

Tree

给定一个 n 个结点 m 条带权边的无向连通图，对于每一条边求出强制包含该边的最小生成树的权值。

$n, m \leq 10^6$

- 先求出原图的最小生成树 T，考虑强制加入边 $e_1 = (u_1, v_1, w_1)$ 后会有怎样的变化。
  1. 若 e_1 已在 T 中：无变化。

例题 2

Tree

给定一个 n 个结点 m 条带权边的无向连通图，对于每一条边求出强制包含该边的最小生成树的权值。

$n, m \leq 10^6$

- 先求出原图的最小生成树 T，考虑强制加入边 $e_1 = (u_1, v_1, w_1)$ 后会有怎样的变化。
  1. 若 e_1 已在 T 中：无变化。
  2. 若 e_1 不在 T 中：问题相当于在最开始先加入边 e_1 再对剩余的边进行 Kruskal 算法，不难发现这对于 T 上不属于 u_1 到 v_1 路径上的边没有任何影响，而且必定存在边 $e_2 = (u_2, v_2, w_2)$ 满足 e_2 在 u_1 到 v_1 路径上，且 $w_2 \leq w_1$，使得求 T 过程中加入 e_2 后 u_1 与 v_1 连通。这样的 e_2 肯定在 Kruskal 算法中是 u_1 到 v_1 路径上最后加入 T 的，即 T 中 u_1 到 v_1 路径上权值最大的边，用 e_1 替换 e_2 即为新的最小生成树。

例题 2

Tree

给定一个 n 个结点 m 条带权边的无向连通图，对于每一条边求出强制包含该边的最小生成树的权值。

$n, m \leq 10^6$

- 先求出原图的最小生成树 T，考虑强制加入边 $e_1 = (u_1, v_1, w_1)$ 后会有怎样的变化。
  1. 若 e_1 已在 T 中：无变化。
  2. 若 e_1 不在 T 中：问题相当于在最开始先加入边 e_1 再对剩余的边进行 Kruskal 算法，不难发现这对于 T 上不属于 u_1 到 v_1 路径上的边没有任何影响，而且必定存在边 $e_2 = (u_2, v_2, w_2)$ 满足 e_2 在 u_1 到 v_1 路径上，且 $w_2 \leq w_1$，使得求 T 过程中加入 e_2 后 u_1 与 v_1 连通。这样的 e_2 肯定在 Kruskal 算法中是 u_1 到 v_1 路径上最后加入 T 的，即 T 中 u_1 到 v_1 路径上权值最大的边，用 e_1 替换 e_2 即为新的最小生成树。
- 问题再次转化成了多次询问树上路径最值问题。

例题 3

动态图连通性

给定一个 n 个结点 m 条边的无向图，你需要对其执行 q 次操作：

1. 加入一条无向边 (u, v)。
2. 删除一条无向边 (u, v)，保证这样的边一定存在。
3. 询问结点 u 和 v 之间的连通性。

$n, m, q \leq 10^5$，允许离线。

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。
- 询问连通性的时候我们考虑在图上行走，设法让所经路径上 t_i 最小的边尽量大，这其实就是例题 1 中的最大瓶颈路问题。

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。
- 询问连通性的时候我们考虑在图上行走，设法让所经路径上 t_i 最小的边尽量大，这其实就是例题 1 中的最大瓶颈路问题。
- 所以我们只需要动态维护整张图按删除时间为权值的最大生成树，就可以实现题目所要求的操作。

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。
- 询问连通性的时候我们考虑在图上行走，设法让所经路径上 t_i 最小的边尽量大，这其实就是例题 1 中的最大瓶颈路问题。
- 所以我们只需要动态维护整张图按删除时间为权值的最大生成树，就可以实现题目所要求的操作。
- 考虑使用 Link Cut Tree 来动态维护无向图的最大生成树：

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。
- 询问连通性的时候我们考虑在图上行走，设法让所经路径上 t_i 最小的边尽量大，这其实就是例题 1 中的最大瓶颈路问题。
- 所以我们只需要动态维护整张图按删除时间为权值的最大生成树，就可以实现题目所要求的操作。
- 考虑使用 Link Cut Tree 来动态维护无向图的最大生成树：
- 当加入一条边 (u, v, t_i) 的时候先判断 u 和 v 是否连通，若不连通直接加入。否则找到 u 到 v 路径上最小的 t_j，若 $t_i > t_j$ 则用新边替换旧边。

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。
- 询问连通性的时候我们考虑在图上行走，设法让所经路径上 t_i 最小的边尽量大，这其实就是例题 1 中的最大瓶颈路问题。
- 所以我们只需要动态维护整张图按删除时间为权值的最大生成树，就可以实现题目所要求的操作。
- 考虑使用 Link Cut Tree 来动态维护无向图的最大生成树：
- 当加入一条边 (u, v, t_i) 的时候先判断 u 和 v 是否连通，若不连通直接加入。否则找到 u 到 v 路径上最小的 t_j，若 $t_i > t_j$ 则用新边替换旧边。
- 当删除一条边 (u, v, t_i) 的时候判断该边是否在当前最大生成树中，若在则将其删除。

- 由于问题离线，我们可以预处理每一条边被删除的时刻，记为 t_i。
- 询问连通性的时候我们考虑在图上行走，设法让所经路径上 t_i 最小的边尽量大，这其实就是例题 1 中的最大瓶颈路问题。
- 所以我们只需要动态维护整张图按删除时间为权值的最大生成树，就可以实现题目所要求的操作。
- 考虑使用 Link Cut Tree 来动态维护无向图的最大生成树：
- 当加入一条边 (u, v, t_i) 的时候先判断 u 和 v 是否连通，若不连通直接加入。否则找到 u 到 v 路径上最小的 t_j，若 $t_i > t_j$ 则用新边替换旧边。
- 当删除一条边 (u, v, t_i) 的时候判断该边是否在当前最大生成树中，若在则将其删除。
- 当询问 u 和 v 连通性时，直接查询 u 和 v 在最大生成树上的连通性即可。
- 时间复杂度为 $O((m + q) \log n)$。

- 第二种方法是线段树分治，在每条边出现的时间区间上添加标记，用按秩合并的并查集（支持撤销操作）处理即可。

- 第二种方法是线段树分治，在每条边出现的时间区间上添加标记，用按秩合并的并查集（支持撤销操作）处理即可。
- 时间复杂度为 $O((m+q)\log^2 n)$。

例题 4

Phone Call（2017 Multi-University Training Contest）

给定一棵 n 个结点的无根树，你需要根据 m 条线路重构一棵生成树。设 $S(u, v)$ 表示包含树上 u 到 v 路径上的所有结点的集合，其中第 i 条线路可以用五元组 $(a_i, b_i, c_i, d_i, w_i)$ 来表达：

- 你可以任意在 $S(a_i, b_i) \cup S(c_i, d_i)$ 的结点之间添加边，每一条的费用都是 w_i。

求总费用之和最小的方案，保证题目有解。

$n, m \leq 10^5$

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。
- 当考虑到第 i 条线路时，我们的目标是让 $S(a_i, b_i) \cup S(c_i, d_i)$ 的所有结点彼此连通。

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。
- 当考虑到第 i 条线路时，我们的目标是让 $S(a_i, b_i) \cup S(c_i, d_i)$ 的所有结点彼此连通。
- 我们可以将其拆成 5 组独立的集合，分别为 $S(a_i, lca(a_i, b_i))$、$S(b_i, lca(a_i, b_i))$、$S(c_i, lca(c_i, d_i))$、$S(d_i, lca(c_i, d_i))$、$\{a_i, c_i\}$，只要使得这 5 组集合分别内部连通就可以达成目标。

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。
- 当考虑到第 i 条线路时，我们的目标是让 $S(a_i, b_i) \cup S(c_i, d_i)$ 的所有结点彼此连通。
- 我们可以将其拆成 5 组独立的集合，分别为 $S(a_i, lca(a_i, b_i))$、$S(b_i, lca(a_i, b_i))$、$S(c_i, lca(c_i, d_i))$、$S(d_i, lca(c_i, d_i))$、$\{a_i, c_i\}$，只要使得这 5 组集合分别内部连通就可以达成目标。
- 经过一些观察我们发现前 4 组的集合都是一段树上的“直链”，当我们要合并 $S(a, b)$ 时，若存在 $S(c, d)$ 已经被合并，我们就可以直接跳过这些结点。

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。
- 当考虑到第 i 条线路时，我们的目标是让 $S(a_i, b_i) \cup S(c_i, d_i)$ 的所有结点彼此连通。
- 我们可以将其拆成 5 组独立的集合，分别为 $S(a_i, lca(a_i, b_i))$、$S(b_i, lca(a_i, b_i))$、$S(c_i, lca(c_i, d_i))$、$S(d_i, lca(c_i, d_i))$、$\{a_i, c_i\}$，只要使得这 5 组集合分别内部连通就可以达成目标。
- 经过一些观察我们发现前 4 组的集合都是一段树上的“直链”，当我们要合并 $S(a, b)$ 时，若存在 $S(c, d)$ 已经被合并，我们就可以直接跳过这些结点。
- 我们使用一个并查集来维护这些“直链”，设 pa_i 表示 i 号结点向上最远的祖先 c 满足 $S(c, i)$ 包含的所有结点均已被合并，初始将所有 pa_i 均设为 i。

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。
- 当考虑到第 i 条线路时，我们的目标是让 $S(a_i, b_i) \cup S(c_i, d_i)$ 的所有结点彼此连通。
- 我们可以将其拆成 5 组独立的集合，分别为 $S(a_i, lca(a_i, b_i))$、$S(b_i, lca(a_i, b_i))$、$S(c_i, lca(c_i, d_i))$、$S(d_i, lca(c_i, d_i))$、$\{a_i, c_i\}$，只要使得这 5 组集合分别内部连通就可以达成目标。
- 经过一些观察我们发现前 4 组的集合都是一段树上的“直链”，当我们要合并 $S(a, b)$ 时，若存在 $S(c, d)$ 已经被合并，我们就可以直接跳过这些结点。
- 我们使用一个并查集来维护这些“直链”，设 pa_i 表示 i 号结点向上最远的祖先 c 满足 $S(c, i)$ 包含的所有结点均已被合并，初始将所有 pa_i 均设为 i。
- 这样我们在合并直链时就可以从底向上依次合并，借助 pa_i 快速找到所有需要合并的“父子关系”，并同时更新 pa_i。

- 我们考虑沿用 Kruskal 算法，先将所有线路按费用从小到大排序，然后逐个考虑每条线路。
- 当考虑到第 i 条线路时，我们的目标是让 $S(a_i, b_i) \cup S(c_i, d_i)$ 的所有结点彼此连通。
- 我们可以将其拆成 5 组独立的集合，分别为 $S(a_i, lca(a_i, b_i))$、$S(b_i, lca(a_i, b_i))$、$S(c_i, lca(c_i, d_i))$、$S(d_i, lca(c_i, d_i))$、$\{a_i, c_i\}$，只要使得这 5 组集合分别内部连通就可以达成目标。
- 经过一些观察我们发现前 4 组的集合都是一段树上的“直链”，当我们要合并 $S(a, b)$ 时，若存在 $S(c, d)$ 已经被合并，我们就可以直接跳过这些结点。
- 我们使用一个并查集来维护这些“直链”，设 pa_i 表示 i 号结点向上最远的祖先 c 满足 $S(c, i)$ 包含的所有结点均已被合并，初始将所有 pa_i 均设为 i。
- 这样我们在合并直链时就可以从底向上依次合并，借助 pa_i 快速找到所有需要合并的“父子关系”，并同时更新 pa_i。
- 由于每个结点只会参与 1 次合并，时间复杂度为 $O((n+m)\log n)$。

例题 5

线路规划（原创）

给定一棵 n 个结点的有根树，1 号节点为根节点。你需要根据 m 条线路重构一棵生成树。

其中第 i 条线路可以用四元组 (x_i, y_i, k_i, w_i) 来表达：

- 你可以在 x_i 和 y_i、x_i 的父亲和 y_i 的父亲、……、x_i 的 $k_i - 1$ 级祖先和 y_i 的 $k_i - 1$ 级祖先之间任意添加边，每一条的费用都是 w_i。

求总费用之和最小的方案，保证题目有解且 x_i 和 y_i 均至少有 $k_i - 1$ 个祖先。

$n, m \leq 152501$

- 先从简单情况入手，考虑解决链上问题。

- 先从简单情况入手，考虑解决链上问题。
- 按照 Kruskal 算法先将边权进行排序，那么我们需要做的其实是这样一件事情：将区间 $[l_1, r_1]$ 里每一个结点向区间 $[l_2, r_2]$ 里对应的结点连边。

- 先从简单情况入手，考虑解决链上问题。
- 按照 Kruskal 算法先将边权进行排序，那么我们需要做的其实是这样一件事情：将区间 $[l_1, r_1]$ 里每一个结点向区间 $[l_2, r_2]$ 里对应的结点连边。
- 考虑使用倍增 ST 表的思想，维护 $\log n$ 层并查集。其中第 i 层的并查集 $pa_i(x)$ 维护的是以 x 为起点的长度为 2^i 的一段区间的连通信息。

- 先从简单情况入手，考虑解决链上问题。
- 按照 Kruskal 算法先将边权进行排序，那么我们需要做的其实是这样一件事情：将区间 $[l_1, r_1]$ 里每一个结点向区间 $[l_2, r_2]$ 里对应的结点连边。
- 考虑使用倍增 ST 表的思想，维护 $\log n$ 层并查集。其中第 i 层的并查集 $pa_i(x)$ 维护的是以 x 为起点的长度为 2^i 的一段区间的连通信息。
- 当我们合并一对长度为 2^i 的区间 $[x, x+2^i-1]$ 与 $[y, y+2^i-1]$ 时，先检查 $pa_i(x)$ 是否已经和 $pa_i(y)$ 连通，否则将 $pa_i(x)$ 与 $pa_i(y)$ 合并，并且暴力递归第 $i-1$ 层的 (x, y) 和 $(x+2^{i-1}, y+2^{i-1})$ 进行合并。这样的合并显然至多会发生 $O(n \log n)$ 次。

- 这样在合并一对普通区间时可以类似 RMQ 问题将其拆成两对长度为 2 的整数次幂的区间，问题就能得到解决。时间复杂度为 $O(m + n \log n \alpha(n))$。

- 这样在合并一对普通区间时可以类似 RMQ 问题将其拆成两对长度为 2 的整数次幂的区间，问题就能得到解决。时间复杂度为 $O(m + n \log n\alpha(n))$。
- 树上问题只需要使用树链剖分将两条路径分解成 $O(\log n)$ 对 DFS 序上的区间，套用链上算法即可。总时间复杂度为 $O(m \log n + n \log n\alpha(n))$。

Break

把一个圆上取 $2n$ 个等分点，并在它们之间两两连边形成一个 $2n$ 阶完全图。问对于任意的正整数 n，是否存在一个哈密顿回路使得图中经过的边在圆上顶点的连线互不平行？

例题 6

开车旅行

某地区的地图可以看成是一个 n 个点 m 条带权边的无向图，其中有 k 个点是加油站，而汽车在加油站可以把油箱加满。已知 x 单位的油箱在不加油的情况下最多能走 x 单位的路程。

现在给定 q 组询问，每次询问能否开一辆油箱容量为 v 单位的汽车从点 x 走到点 y。

$k, n, q \leq 200000$，保证所有点 x、y 均为加油站。

- 我们如果能求出加油站之间的最小生成树，问题就转化成了多次询问最小瓶颈路问题。

- 我们如果能求出加油站之间的最小生成树，问题就转化成了多次询问最小瓶颈路问题。
- 一种可行的做法是把每个加油站均作为起点计算多源最短路，并同时记录每个点距离最近的加油站编号。

- 我们如果能求出加油站之间的最小生成树，问题就转化成了多次询问最小瓶颈路问题。
- 一种可行的做法是把每个加油站均作为起点计算多源最短路，并同时记录每个点距离最近的加油站编号。
- 然后遍历每条边 (u, v, w)，如果两端点距离最近的加油站编号不同，则向这两个编号的加油站之间连一条长度为 $dist_u + dist_v + w$ 的边，然后在得到的新图上求出最小生成树即可。

- 我们如果能求出加油站之间的最小生成树，问题就转化成了多次询问最小瓶颈路问题。
- 一种可行的做法是把每个加油站均作为起点计算多源最短路，并同时记录每个点距离最近的加油站编号。
- 然后遍历每条边 (u, v, w)，如果两端点距离最近的加油站编号不同，则向这两个编号的加油站之间连一条长度为 $dist_u + dist_v + w$ 的边，然后在得到的新图上求出最小生成树即可。
- 时间复杂度为 $O((n + m + q) \log n)$。

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。
- 若 x 到 y 路径上存在一条边 (u, v, w) 满足距离 u 最近的加油站为 x 且距离 v 最近的加油站为 y，那么我们会将 $(x, y, dist(x, y))$ 加入生成树。

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。
- 若 x 到 y 路径上存在一条边 (u, v, w) 满足距离 u 最近的加油站为 x 且距离 v 最近的加油站为 y，那么我们会将 $(x, y, dist(x, y))$ 加入生成树。
- 否则我们可以证明 x 到 y 的路径不可能在最小生成树中出现：

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。
- 若 x 到 y 路径上存在一条边 (u, v, w) 满足距离 u 最近的加油站为 x 且距离 v 最近的加油站为 y，那么我们会将 $(x, y, dist(x, y))$ 加入生成树。
- 否则我们可以证明 x 到 y 的路径不可能在最小生成树中出现：
  1. 找到一条边 (u, v, w)，设距离 u 最近的加油站为 a，距离 v 最近的加油站为 b，u 距离 x 近，v 距离 y 近。

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。
- 若 x 到 y 路径上存在一条边 (u, v, w) 满足距离 u 最近的加油站为 x 且距离 v 最近的加油站为 y，那么我们会将 $(x, y, dist(x, y))$ 加入生成树。
- 否则我们可以证明 x 到 y 的路径不可能在最小生成树中出现：
  1. 找到一条边 (u, v, w)，设距离 u 最近的加油站为 a，距离 v 最近的加油站为 b，u 距离 x 近，v 距离 y 近。
  2. 由条件可得 $dist(a, u) \leq dist(x, u)$，$dist(b, v) \leq dist(y, v)$。

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。
- 若 x 到 y 路径上存在一条边 (u, v, w) 满足距离 u 最近的加油站为 x 且距离 v 最近的加油站为 y，那么我们会将 $(x, y, dist(x, y))$ 加入生成树。
- 否则我们可以证明 x 到 y 的路径不可能在最小生成树中出现：
  1. 找到一条边 (u, v, w)，设距离 u 最近的加油站为 a，距离 v 最近的加油站为 b，u 距离 x 近，v 距离 y 近。
  2. 由条件可得 $dist(a, u) \leq dist(x, u)$，$dist(b, v) \leq dist(y, v)$。
  3. 因为 $dist(a, b) = dist(a, u) + dist(v, b) + w \leq dist(x, b) = dist(x, u) + dist(v, b) + w, dist(a, y) = dist(a, u) + dist(v, y) + w \leq dist(x, y) = dist(x, u) + dist(v, y) + w$，
  4. 所以直接从 x 到 y 不会比从 x 到 b 再从 b 到 a 最后从 a 到 y 优。

证明

- 考虑从加油站 x 出发可行的路线，肯定是走最短路径到达另一个加油站 y。
- 若 x 到 y 路径上存在一条边 (u, v, w) 满足距离 u 最近的加油站为 x 且距离 v 最近的加油站为 y，那么我们会将 $(x, y, dist(x, y))$ 加入生成树。
- 否则我们可以证明 x 到 y 的路径不可能在最小生成树中出现：
  1. 找到一条边 (u, v, w)，设距离 u 最近的加油站为 a，距离 v 最近的加油站为 b，u 距离 x 近，v 距离 y 近。
  2. 由条件可得 $dist(a, u) \leq dist(x, u)$，$dist(b, v) \leq dist(y, v)$。
  3. 因为 $dist(a, b) = dist(a, u) + dist(v, b) + w \leq dist(x, b) = dist(x, u) + dist(v, b) + w, dist(a, y) = dist(a, u) + dist(v, y) + w \leq dist(x, y) = dist(x, u) + dist(v, y) + w$，
  4. 所以直接从 x 到 y 不会比从 x 到 b 再从 b 到 a 最后从 a 到 y 优。
- 证毕。

例题 7

Peaks (BZOJ3551)

在 Bytemountains 有 n 座山峰，每座山峰有他的高度 h_i。有些山峰之间有双向道路相连，共 m 条道路，每条道路有一个困难值，这个值越大表示越难走。

现在有 q 组询问，每组询问询问从点 v 开始只经过困难值小于等于 x 的路径所能到达的山峰中第 k 高的山峰，如果无解输出 -1。

$n \leq 10^5$，$m, q \leq 5 \times 10^5$，强制在线。

- 如果问题允许离线，我们可以把道路和询问混在一起排序，进行 Kruskal 算法的同时用平衡树的合并维护答案。

- 如果问题允许离线，我们可以把道路和询问混在一起排序，进行 Kruskal 算法的同时用平衡树的合并维护答案。
- 要求在线算法的话我们可以改造生成树，在添加边 (u, v, w) 时不直接将 u 在并查集上的根 x 父亲设为 v 在并查集上的根 y，而是新建结点 z 并连接 $z \to x$ 和 $z \to y$，同时将 z 的权值赋为 w。

- 如果问题允许离线，我们可以把道路和询问混在一起排序，进行 Kruskal 算法的同时用平衡树的合并维护答案。
- 要求在线算法的话我们可以改造生成树，在添加边 (u, v, w) 时不直接将 u 在并查集上的根 x 父亲设为 v 在并查集上的根 y，而是新建结点 z 并连接 $z \to x$ 和 $z \to y$，同时将 z 的权值赋为 w。
- 这样我们发现从 v 开始只经过权值小于等于 x 的路径能走到的结点集合就对应生成树上的一棵子树，其根节点为距离 v 最远的权值不超过 x 的结点。

- 如果问题允许离线，我们可以把道路和询问混在一起排序，进行 Kruskal 算法的同时用平衡树的合并维护答案。
- 要求在线算法的话我们可以改造生成树，在添加边 (u, v, w) 时不直接将 u 在并查集上的根 x 父亲设为 v 在并查集上的根 y，而是新建结点 z 并连接 $z \to x$ 和 $z \to y$，同时将 z 的权值赋为 w。
- 这样我们发现从 v 开始只经过权值小于等于 x 的路径能走到的结点集合就对应生成树上的一棵子树，其根节点为距离 v 最远的权值不超过 x 的结点。
- 使用 DFS 序和可持久化线段树维护答案信息，就可以在 $O((n+m)\log n) - O(\log n)$ 的时间复杂度解决该问题。

例题 8

XOR

给定一个 n 阶完全无向图，每个节点有权值 v_i。

对于任意结点 x、y，从 x 到 y 的无向边权值为 $v_x \bigoplus v_y$，求最小生成树。

$n \leq 10^5$，$v_i \leq 10^9$。

例题 8

XOR

给定一个 n 阶完全无向图，每个节点有权值 v_i。

对于任意结点 x、y，从 x 到 y 的无向边权值为 $v_x \bigoplus v_y$，求最小生成树。

$n \leq 10^5$，$v_i \leq 10^9$。

这种特殊的位运算生成树问题，一般需要用到最小生成树的第三种算法——Boruvka 算法。

Boruvka 算法

- Step1：对于当前每个连通块，找到其连出的权值最小的出边。
- Step2：将所有这样的出边加入最小生成树（需要去重），并合并相应的连通块。
- Step3：重复 Step1、Step2 直到仅剩 1 个连通块。由于每轮连通块个数至少减半，最多进行 $O(\log n)$ 轮。
- 时间复杂度为 $O(m \log n)$。

Boruvka 算法

- Step1：对于当前每个连通块，找到其连出的权值最小的出边。
- Step2：将所有这样的出边加入最小生成树（需要去重），并合并相应的连通块。
- Step3：重复 Step1、Step2 直到仅剩 1 个连通块。由于每轮连通块个数至少减半，最多进行 $O(\log n)$ 轮。
- 时间复杂度为 $O(m \log n)$。
- 那么我们现在面临的问题就是这样：有 n 个数，每个数有一个颜色，对于每个数 a 求出一个异色的数 b，使得 $a \bigoplus b$ 最小。

Boruvka 算法

- Step1：对于当前每个连通块，找到其连出的权值最小的出边。
- Step2：将所有这样的出边加入最小生成树（需要去重），并合并相应的连通块。
- Step3：重复 Step1、Step2 直到仅剩 1 个连通块。由于每轮连通块个数至少减半，最多进行 $O(\log n)$ 轮。
- 时间复杂度为 $O(m \log n)$。
- 那么我们现在面临的问题就是这样：有 n 个数，每个数有一个颜色，对于每个数 a 求出一个异色的数 b，使得 $a \bigoplus b$ 最小。
- 对每个数按二进制从大到小建立 Trie 树，维护一下子树内包含颜色的最大最小值就可以知道是否存在被询问颜色外的其他颜色，从而轻松求出答案。

Boruvka 算法

- Step1：对于当前每个连通块，找到其连出的权值最小的出边。
- Step2：将所有这样的出边加入最小生成树（需要去重），并合并相应的连通块。
- Step3：重复 Step1、Step2 直到仅剩 1 个连通块。由于每轮连通块个数至少减半，最多进行 $O(\log n)$ 轮。
- 时间复杂度为 $O(m \log n)$。
- 那么我们现在面临的问题就是这样：有 n 个数，每个数有一个颜色，对于每个数 a 求出一个异色的数 b，使得 $a \bigoplus b$ 最小。
- 对每个数按二进制从大到小建立 Trie 树，维护一下子树内包含颜色的最大最小值就可以知道是否存在被询问颜色外的其他颜色，从而轻松求出答案。

总时间复杂度为 $O(n \log n \log v)$。

例题 9

生成树计数

给定一个 n 阶无向完全图 G，每一条边有一个非负的权值。对于 G 的一个生成树 T，定义

$$F(T,k)=(\sum_{e\in T}w_e)^k$$

求 G 的所有生成树的 $F(T,k)$ 之和，对 998244353 取模。

cont'd

- 我们考虑用生成函数来表示若干个数和的 k 次幂，相当于在这些数中带重复选出 k 个数，把所有选法中数字的乘积加在一起。

$$(\sum_{i=1}^{n} v_i)^k = \sum_{1 \le p_1 \le \cdots \le p_k \le n} (\prod_{i=1}^{k} v_{p_i}) = k! \times \sum_{c_1 + \cdots + c_n = k} (\prod_{i=1}^{n} \frac{v_i^{c_k}}{c_k!})$$

cont'd

- 我们考虑用生成函数来表示若干个数和的 k 次幂，相当于在这些数中带重复选出 k 个数，把所有选法中数字的乘积加在一起。

$$(\sum_{i=1}^{n} v_i)^k = \sum_{1 \le p_1 \le \cdots \le p_k \le n} (\prod_{i=1}^{k} v_{p_i}) = k! \times \sum_{c_1 + \cdots + c_n = k} (\prod_{i=1}^{n} \frac{v_i^{c_k}}{c_k!})$$

- 我们把每条边用

$$G(w) = \sum_{i=0}^{k} \frac{w^i}{i!} x^i$$

代替，那么对于一棵生成树 T，

$$F(T) = \prod_{e \in T} G(w_e)$$

的 k 次方系数再乘上 $i!$ 就是其权值。

- 这样一来用矩阵树定理求出这个多项式矩阵的行列式即为答案，考虑到直接求元素是多项式的矩阵的行列式很麻烦，我们可以先随便代入 $nk+1$ 个不同的 x 求出 $nk+1$ 个点值，最后通过拉格朗日插值还原出行列式。

- 这样一来用矩阵树定理求出这个多项式矩阵的行列式即为答案，考虑到直接求元素是多项式的矩阵的行列式很麻烦，我们可以先随便代入 $nk+1$ 个不同的 x 求出 $nk+1$ 个点值，最后通过拉格朗日插值还原出行列式。
- 时间复杂度为 $O(kn^4)$。

Break

给定一张 100 个点 2500 条边的无向图 G，边从 1 到 2500 编号，保证无自环。

证明：G 中必定存在一条至少包含 50 条边的简单路径，且从起点开始经过的每条边的编号都是递增的。

THX

Q & A